

SMS Spam Detection Using LSTM – Project Documentation – DIVYA N (23AD037)

1. Project Overview

Problem Statement:

SMS spam detection is a critical task in communication security. The goal of this project is to build a deep learning model that classifies SMS messages as either spam or legitimate (ham) using an LSTM neural network.

Key Goals:

- Build an LSTM-based text classification model.
- Handle class imbalance properly.
- Demonstrate understanding of text preprocessing, sequence modeling, and evaluation metrics.

2. Dataset Description

Location: data/spam.csv

Total Samples: 5,572 SMS messages

Columns: v1 (label), v2 (message)

Class	Count	Percentage
Ham	4827	~87%
Spam	747	~13%

Sample Messages:

Type	Example
Ham	Hey, are we still meeting for lunch today?
Spam	Congratulations! You've won a free iPhone. Click here to claim.

Observation: The dataset is imbalanced, with far fewer spam messages than ham messages. Handling this imbalance is crucial for model performance.

3. Data Preprocessing

Steps Implemented:

- Loaded dataset and explored basic stats.
- Cleaned text: removed special characters, URLs, and phone numbers.
- Tokenized messages using Keras tokenizer.
- Converted text to sequences and applied padding (MAX_LEN=100) for uniform input size.
- Handled class imbalance by using class weights in the loss function (pos_weight=3.0) to give higher importance to spam.

Example Code Snippet:

```
from utils.preprocessing import clean_text
from tensorflow.keras.preprocessing.sequence import pad_sequences

cleaned = clean_text(message)
seq = tokenizer.texts_to_sequences([cleaned])
padded = pad_sequences(seq, maxlen=MAX_LEN, padding='post')
```

4. Model Architecture

SpamDetector (LSTM)

Layer	Details
Embedding	VOCAB_SIZE=5000, embedding_dim=128
LSTM	64 hidden units
Dropout	0.2
Dense	1 output unit with sigmoid activation

Loss Function: BCEWithLogitsLoss with pos_weight=3.0

Optimizer: Adam

Rationale:

- LSTM captures sequential dependencies in text.
- Dropout prevents overfitting.
- Weighted loss addresses spam class imbalance.

5. Training Pipeline

Configuration:

- Epochs: 20
- Batch size: 32
- Early stopping: patience = 3

Training Steps:

- Load preprocessed data and class weights.
- Forward pass through LSTM.
- Compute weighted binary cross-entropy loss.
- Backpropagation with Adam optimizer.
- Save best model (best_model.pt) based on validation loss.

6. Evaluation

Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score (especially for spam class)
- Confusion matrix

Threshold Adjustment: To increase spam recall, prediction threshold set to 0.4.

Class	Precision	Recall	F1-score	Support
Ham	0.98	0.97	0.97	966
Spam	0.82	0.84	0.83	149

Accuracy: 95%

Observation: Model detects most ham messages correctly; spam detection can improve further with threshold tuning and class weights.

7. Streamlit UI

Features:

- Input box for SMS message.
- Predict button to classify message as Ham or Spam.
- Shows confidence score.
- Threshold aligned with evaluation: 0.4 for spam.

Screenshot Example: (Insert screenshots of app with Ham and Spam predictions here)

8. Output Screenshots:

```
VYA
[nltk_data]      N\AppData\Roaming\nltk_data...
[nltk_data]      Package stopwords is already up-to-date!
precision    recall  f1-score   support

      Ham      0.93      0.99      0.96       966
      Spam      0.90      0.52      0.66       149

 accuracy              0.93       1115
 precision    recall  f1-score   support

      Ham      0.93      0.99      0.96       966
      Spam      0.90      0.52      0.66       149

 accuracy              0.93       1115
      Ham      0.93      0.99      0.96       966
      Spam      0.90      0.52      0.66       149

 accuracy              0.93       1115
      Spam      0.90      0.52      0.66       149

 accuracy              0.93       1115

 accuracy              0.93       1115
 accuracy              0.93       1115
 macro avg      0.91      0.76      0.81       1115
 macro avg      0.91      0.76      0.81       1115
 weighted avg    0.93      0.93      0.92       1115
 weighted avg    0.93      0.93      0.92       1115

PS D:\final ak> streamlit run app.py
```



SMS Spam Detection System

Enter a message below to check whether it is Spam or Ham.

Enter SMS Message

Hey, are we meeting at 5pm?

Predict

✓ Legitimate (Ham) Message

Confidence: 0.88



SMS Spam Detection System

Enter a message below to check whether it is Spam or Ham.

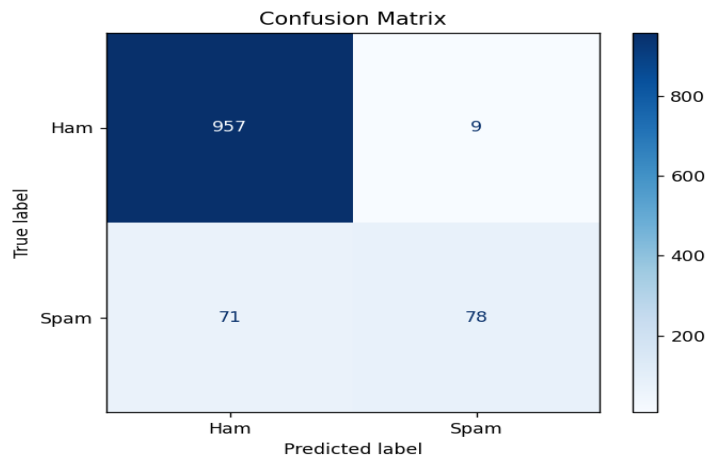
Enter SMS Message

Congratulations! You won a free iPhone. Click here now!

Predict

🚨 Spam Detected

Confidence: 0.35



9. Folder Structure

```
├── requirements.txt
├── data/
│   └── spam.csv
├── model/
│   ├── spam_detector.py
│   └── data_loader.py
├── utils/
│   ├── preprocessing.py
│   ├── metrics.py
│   └── class_weights.py
├── train.py
├── evaluate.py
└── app.py
```

To Run:

Install dependencies:

```
pip install -r requirements.txt
```

Train the model:

```
python train.py
```

Evaluate performance:

```
python evaluate.py
```

Run Streamlit UI:

```
streamlit run app.py
```

10. Key Learnings

- Preprocessing text is crucial for LSTM performance.
- Class imbalance drastically affects spam F1-score.
- Threshold tuning improves recall.
- LSTM captures sequential patterns in messages.
- Streamlit provides easy deployment for interactive testing.